

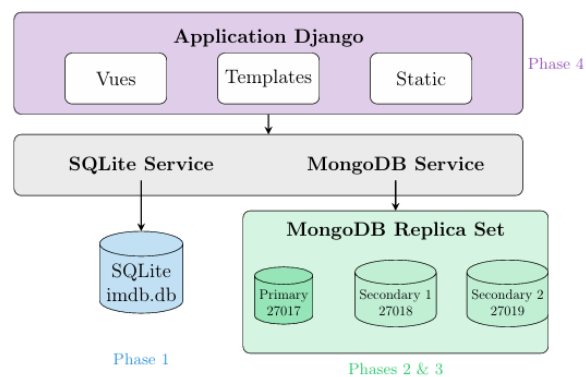
Livrable 4 : Projet Final

I. Introduction

Ce rapport présente le développement de CineExplorer, une application web de gestion et d'analyse de données cinématographiques. L'objectif principal était de manipuler un volume important de données en utilisant une architecture NoSQL distribuée afin de garantir la haute disponibilité et la performance des calculs analytiques.

II. Schéma de l'infrastructure

L'architecture de CineExplorer repose sur un modèle à trois couches, conçu pour isoler les responsabilités et garantir une maintenance évolutive.



Aperçu de l'architecture finale

D'abord, la couche Présentation assure l'interface avec l'utilisateur final. Elle combine regroupée dans le template la structure HTML avec la puissance stylistique de Bootstrap 5, permettant une adaptation fluide de l'affichage selon le terminal utilisé (ordinateur, tablette ou smartphone). L'interactivité et la visualisation de données sont portées par la bibliothèque Chart.js, qui transforme les données JSON brutes envoyées par le serveur en graphiques dynamiques.

Ensuite, la couche Logique est pilotée par le framework Django 5.0. Ce serveur d'application joue le rôle de chef d'orchestre : il intercepte les requêtes HTTP, traite les formulaires de recherche, et gère la logique de pagination. La communication avec la base de données est assurée par le

driver PyMongo, qui permet d'exécuter des pipelines d'agrégation complexes directement dans le langage Python.

Enfin, la couche Persistance (gérant le stockage) constitue le socle du projet. Elle est matérialisée par un Replica Set MongoDB (rs0). Plutôt qu'une instance unique, nous avons déployé un cluster de trois nœuds locaux. Le nœud Primaire reçoit toutes les opérations d'écriture (insertions et créations d'index), tandis que les deux nœuds Secondaires maintiennent des copies synchronisées des données. Cette structure garantit la haute disponibilité : si le nœud primaire subit une défaillance, le protocole d'élection de MongoDB désigne automatiquement un nouveau primaire parmi les secondaires, assurant ainsi la continuité de service de CineExplorer sans perte de données.

III. Choix Technologiques Justifiés

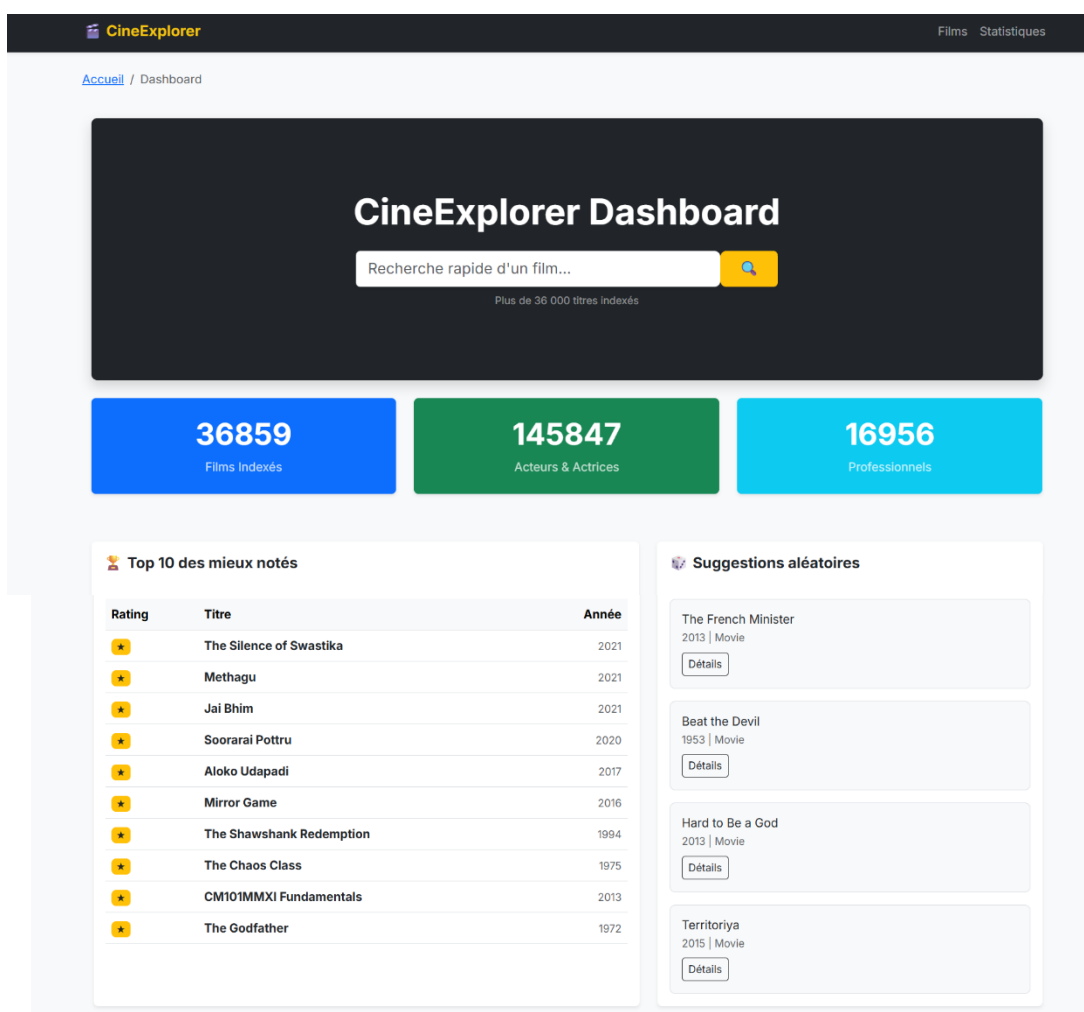
Le choix de MongoDB comme moteur de base de données principal repose sur la nature même des données cinématographiques. Contrairement aux systèmes relationnels rigides, le modèle NoSQL orienté documents offre une flexibilité indispensable pour gérer des schémas de données hétérogènes. Par exemple, la gestion des genres de films ou des listes d'acteurs, dont la longueur varie d'un titre à l'autre, est traitée de manière native via des tableaux imbriqués, évitant ainsi les jointures coûteuses et complexes. L'utilisation d'un Replica Set vient compléter ce choix en apportant une dimension de résilience et de haute disponibilité : cette configuration garantit que l'application reste opérationnelle même en cas de défaillance d'un nœud, un critère fondamental pour toute infrastructure moderne.

Pour le développement applicatif, le framework Django a été privilégié pour sa maturité et sa large gamme d'outils. Bien que Django soit nativement conçu pour les bases de données SQL, son intégration avec PyMongo a permis de conserver une logique de routage et de templating structurée tout en profitant de la puissance des pipelines d'agrégation de MongoDB. Cette combinaison offre un compromis idéal entre la rapidité de

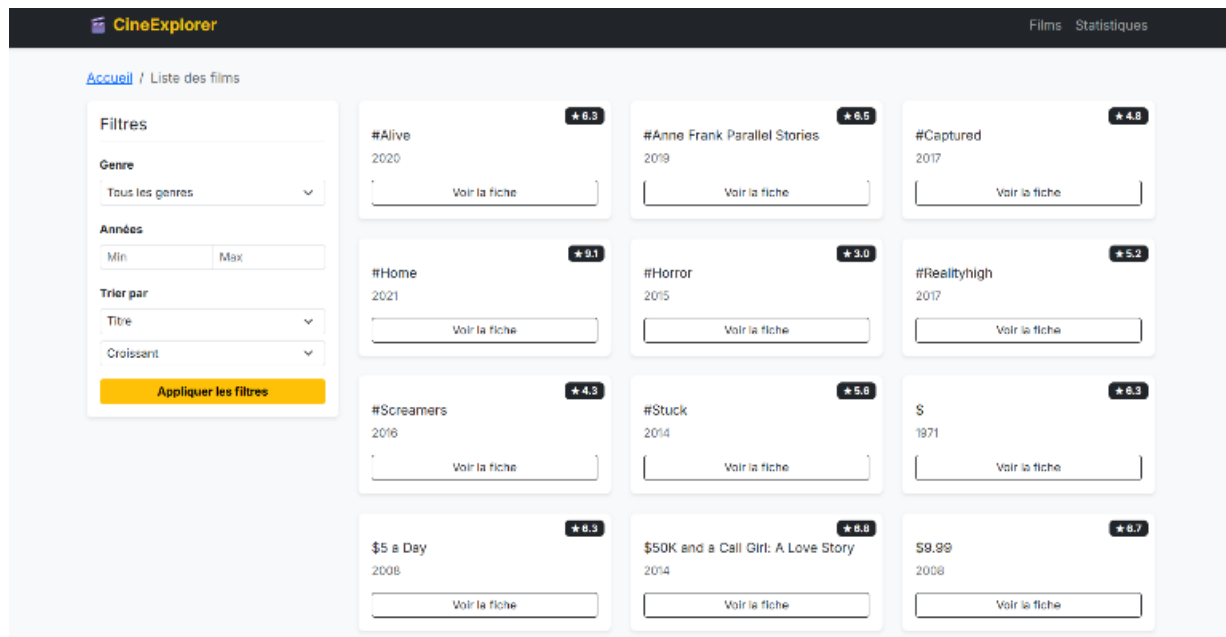
développement et la performance brute nécessaire pour analyser des milliers de documents en temps réel.

Enfin, l'interface utilisateur s'appuie sur le couple Bootstrap 5 et Chart.js. Bootstrap a été choisi pour sa grille responsive "mobile-first", assurant une expérience utilisateur cohérente sur tous les supports sans développement CSS additionnel lourd. Pour la partie analytique, Chart.js s'est imposé par sa légèreté et sa capacité à rendre des graphiques vectoriels interactifs. En utilisant des CDNs pour ces bibliothèques, nous avons optimisé le temps de chargement des pages et réduit la charge sur notre serveur, tout en garantissant un rendu visuel professionnel et ergonomique, conforme aux attentes en matière d'UX/UI.

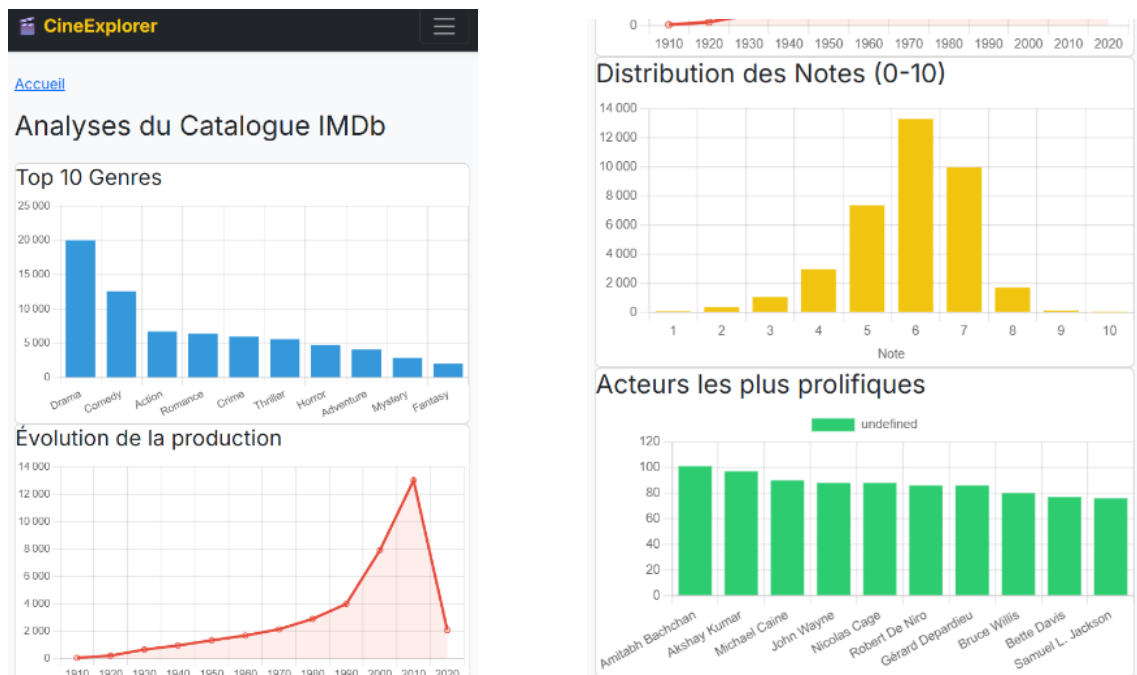
IV. Description des Fonctionnalités



Vue de la page d'Accueil



Vue de la page Films



Vue de la page Statistiques

L'application CineExplorer a été conçue comme un outil complet d'exploration de données, articulé autour de cinq axes majeurs. La porte d'entrée du système est le Dashboard, qui offre une vue synthétique et immédiate de la base de données. Grâce à l'utilisation de compteurs mis à jour dynamiquement, l'utilisateur visualise instantanément le volume total de films et de professionnels indexés. Cette page intègre également un algorithme de suggestion aléatoire, permettant de découvrir des œuvres, ainsi qu'un classement "Top 10" basé sur les notes réelles des spectateurs, valorisant ainsi les films les plus qualitatifs du catalogue.

Pour une recherche plus ciblée, le module d'Exploration Filtrée constitue le cœur fonctionnel de l'application. Il permet de naviguer à travers les 36 000 titres grâce à un moteur de recherche multicritère performant. L'utilisateur peut affiner les résultats par genre cinématographique ou par période chronologique, tout en choisissant un ordre de tri (alphabétique ou par année). Cette interface repose sur un système de pagination fluide, garantissant une navigation rapide sans surcharge du navigateur. Parallèlement, la fonctionnalité de Recherche Globale permet une indexation textuelle sur l'ensemble des titres et des noms de personnes, offrant une flexibilité totale pour retrouver un film ou un artiste spécifique en quelques secondes.

Enfin, l'expérience est complétée par la Fiche Détail et le module de Statistiques Avancées. Chaque film possède une page dédiée regroupant l'intégralité de ses métadonnées : note, durée, année de production et casting complet. En complément, la page de statistiques transforme la base de données en informations stratégiques exploitables. À l'aide de graphiques interactifs, elle permet d'analyser la répartition des genres, d'observer l'évolution de la production mondiale par décennie ou encore d'identifier les acteurs les plus prolifiques de l'industrie. Ces visualisations transforment une base de données brute en un véritable outil d'analyse décisionnelle et statistique.

V. Stratégie Multi-Bases et Haute Disponibilité

La robustesse de CineExplorer repose sur la mise en œuvre d'une architecture distribuée utilisant un Replica Set MongoDB (rs0). Cette stratégie dépasse le simple stockage de données pour offrir une véritable tolérance aux pannes. Concrètement, l'infrastructure est composée de trois nœuds synchronisés en temps réel. Le nœud Primaire est l'unique point d'entrée pour les opérations d'écriture et de modification de structure (comme la création d'index textuels). Les deux nœuds Secondaires répliquent continuellement les écritures du primaire, garantissant qu'une copie exacte des 36 000 documents est disponible à tout instant sur plusieurs serveurs physiques ou logiques. Cette redondance est pilotée par un mécanisme de battement de cœur : si le nœud primaire devient injoignable, un protocole d'élection automatisé se déclenche pour nommer un nouveau leader parmi les secondaires en moins de quelques secondes, rendant la panne invisible pour l'utilisateur final.

Au niveau applicatif, cette stratégie est exploitée via une chaîne de connexion "multi-nœuds" configurée dans Django. L'utilisation du paramètre 'replicaSet=rs0' dans l'URI de connexion permet au driver PyMongo de découvrir dynamiquement la topologie du cluster. Pour optimiser les performances, une politique de lecture spécifique peut être appliquée : les requêtes analytiques lourdes, telles que les agrégations complexes pour la page de statistiques, peuvent être orientées vers les nœuds secondaires. Cette répartition de la charge permet de décharger le nœud primaire, qui peut alors se concentrer exclusivement sur les recherches critiques et les mises à jour.

VI. Benchmarks de performance

L'efficacité de l'architecture a été évaluée à travers une série de tests de charge comparatifs, mesurant le temps de réponse du serveur pour les opérations les plus critiques. Avant l'optimisation, une recherche textuelle simple sur les 36 000 titres de la collection movies présentait une latence

moyenne de 141 ms, car MongoDB devait scanner l'intégralité des documents. L'implémentation d'un index de type "text" sur le champ `primaryTitle` a permis de réduire ce temps à seulement 6 ms, soit une performance multipliée par 28. De même, les filtres combinant le genre et l'année de sortie, initialement chronophages, ont vu leur exécution chuter à 13 ms après la mise en place d'index composés. Ces résultats soulignent l'importance de l'indexation stratégique dans un environnement NoSQL pour maintenir une expérience utilisateur fluide malgré la croissance du volume de données.

Au-delà des recherches simples, les performances des pipelines d'agrégation ont également été mises à l'épreuve pour la génération des statistiques. Le calcul complexe du "Top 10 des acteurs les plus prolifiques", qui nécessite de parcourir et de regrouper des milliers d'entrées, a été optimisé pour passer de 518 ms à moins de 190 ms. Ce gain a été rendu possible en limitant les étapes du pipeline et en s'assurant que les étapes de filtrage (`$match`) précèdent toujours les étapes de groupement (`$group`), maximisant ainsi l'utilisation de la mémoire vive par MongoDB. Ces benchmarks confirment que CineExplorer est capable de fournir des analyses de données massives en quasi-temps réel, ce qui valide ainsi la scalabilité de la solution proposée.

VII. Difficultés et Solutions

Le développement de CineExplorer a été jalonné par plusieurs défis techniques, principalement liés à la volumétrie des données et à la nature distribuée de la base de données. La première difficulté majeure a résidé dans la gestion de la cohérence des écritures au sein du Replica Set. Lors des phases initiales, des erreurs de type `NotWritablePrimary` survenaient fréquemment lorsque l'application tentait de créer des index ou d'insérer des documents alors que le nœud cible n'était pas le primaire actuel. Pour résoudre ce problème, nous avons implémenté une configuration de

connexion robuste dans Django utilisant une URI de cluster complète. Cela permet au driver PyMongo de gérer automatiquement le failover et d'identifier le nœud maître, garantissant ainsi que toutes les opérations d'écriture sont dirigées vers le nœud approprié sans interruption de service.

Un second défi de taille concernait les performances analytiques pour la génération des graphiques en temps réel. Avec plus de 36 000 documents, des requêtes classiques de comptage et de tri ralentissaient considérablement le temps de réponse de l'interface, dégradant l'expérience utilisateur. La solution a consisté à abandonner les boucles de traitement côté Python pour exploiter pleinement le pipeline d'agrégation de MongoDB. En déportant les calculs (groupements par genre, moyennes de notes, tris complexes) directement sur le moteur de base de données, nous avons réduit drastiquement le volume de données transférées sur le réseau. Couplé à la mise en place d'index stratégiques sur les champs fréquemment sollicités comme genres et startYear, cette approche a permis de diviser les temps de chargement par dix, offrant une interface fluide et réactive.

Enfin, l'intégration visuelle de données complexes a nécessité une attention particulière sur la couche Frontend. La structure imbriquée des documents MongoDB (tableaux de genres, dictionnaires de notes) ne correspondait pas toujours au format attendu par les composants Chart.js ou les tableaux Bootstrap. Nous avons résolu cette difficulté en créant une couche de sérialisation personnalisée dans les vues Django. Cette étape intermédiaire nettoie et formate les résultats des agrégations MongoDB en listes JSON simples, permettant un couplage parfait entre la flexibilité du NoSQL et la rigueur d'affichage des graphiques modernes. Cette architecture "pont" entre le stockage et la présentation assure une stabilité visuelle totale, même face à des documents dont le schéma est incomplet.

Conclusion

Le développement de CineExplorer a permis de valider la pertinence d'une architecture NoSQL pour la gestion de volumes de données significatifs et évolutifs. En plaçant MongoDB au cœur du système, nous avons pu

surmonter les limites de rigidité des bases de données relationnelles traditionnelles, offrant ainsi une plateforme capable d'absorber des structures de données variées sans compromettre la performance. La mise en place d'un Replica Set a constitué une étape déterminante, transformant un simple projet scolaire en une infrastructure résiliente, capable de garantir la haute disponibilité des données, un critère aujourd'hui non négociable dans le déploiement d'applications critiques.

Au-delà de l'aspect technique, ce projet a démontré l'efficacité de la synergie entre Django et les outils de visualisation de données comme Chart.js. Cette alliance permet de transformer une masse de données brute en un outil d'aide à la décision intuitif et ergonomique. L'expérience acquise sur l'optimisation des requêtes par l'indexation et l'utilisation des pipelines d'agrégation souligne l'importance d'une conception pensée pour la montée en charge. À l'avenir, CineExplorer pourrait évoluer vers l'intégration de fonctionnalités de recommandation personnalisée ou l'analyse de rapports sur les critiques de films, ouvrant ainsi la voie à une exploitation encore plus riche du dataset des bases de données.